ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



REPORT WEEK 2

Project: ”Design Space Exploration of a Shared L2 Cache with gem5”

Student: Luong Khanh Ngoc

ID: 24200134

TP.Ho Chi Minh, October 2026

1. Summary

Question for P1: When L2 ways are switched off, how much does each workload lose? For every workload the goal is a curve of L2 miss rate against the number of active ways, with the shape explained and checked against gem5.

What was done:

- **Stage 0:** an interim gem5 baseline (4 × RISC-V MinorCPU, shared L2 1 MiB / 16 ways / 64 B lines, LRU, no prefetcher), checked step by step.
- **Oracle v1 (Streaming):** a Python LRU model of L1D + L2. It reproduces gem5 and locates the streaming capacity cliff at **64 KiB × active ways**, confirmed by gem5 at 6 points next to the cliff.
- **Oracle v2 (stack distance):** a single pass per workload gives the miss rate for **every way count from 1 to 16**. All four sweep workloads were profiled. Temporal512K was checked on gem5 at 16/8/4 ways, with a difference below 0.1 percentage points.

Workload	16 ways	12 ways	8 ways	4 ways	Curve shape	Status
loop768K	0%	0%	100%	100%	sharp cliff at 12 ways	oracle; gem5 pending
stream3x2M	100%	100%	100%	100%	flat (already past capacity)	oracle; gem5 pending
temporal512K	9.4%	11.5%	27.2%	63.7%	floor + soft knee near 8 ways	**oracle = gem5**
graph2M	39.6%	53.9%	69.3%	85.4%	almost linear, ~3.7 points/way	oracle; gem5 pending

L2 miss rate by active ways (oracle, Computed). Temporal512K is also Measured on gem5: 9.36 / 27.22 / 63.63%.

Main findings:

1. Streaming has a sharp cliff at footprint = 64 KiB × W. gem5 sits about 1.2% above the oracle exactly at the cliff, because code and stack lines displace data in about 12 sets.
2. Temporal locality has a **floor ≈ 100 − hotpct** that no number of ways can remove, and a **knee ≈ hot / 64 KiB ways**. Both are confirmed by four parameter variants.
3. The gap between oracle and gem5 is not constant: about 1% where the curve jumps (streaming cliff) and below 0.1% where it is smooth (temporal).
4. For P1: temporal512K loses only **+2.1 points** going from 16 to 12 ways, but **+15.7 points** from 12 to 8 ways; streaming larger than L2 loses nothing; loop768K is free down to 12 ways.

2. Stage 0 – Environment and Interim Baseline

The interim baseline is used until the official baseline from Tiến Anh is available. Environment: WSL Ubuntu 24.04, gem5 v25.1.0.1 (RISCV build, classic memory, SE mode), workloads built with `riscv64-linux-gnu-g++ -O2 -static`.

Component	Value	Source / label
CPU	4 × RiscvMinorCPU, 1-issue, 1 GHz	close to in-order CVA6
L1I / L1D	16 KiB 4-way / 32 KiB 8-way (64 sets), 1 MSHR, no prefetcher	latencies Assumed
Interconnect	L2XBar, 2 + 2 cycles, 16 B/cycle	split Assumed
Shared L2	1 MiB = 1024 sets × 16 ways × 64 B, LRU, 32 MSHRs, mostly-inclusive, no prefetcher	1 MiB Assumed
DRAM	DDR4-2400 8x8, 1 channel	spec proposal
Way shutdown	1024 sets kept, W ways active → capacity = 64 KiB × W	`--l2-ways W`

Step	Check	Result
b1	Build the workload for host (x86) and RISC-V	
b2	Hello world + stream 256K; host vs gem5 checksum	PASS
b3	Read config.ini: 1 MiB, 16 ways, 32 MSHRs, LRU, prefetcher Null, 64 B line	all correct
—	Hello world on 4 cores	21,148 instructions, 1,896 L2 accesses
b4	Stream 768K: 16 ways miss 0.02%, 8 ways 100.00%; DRAM reads 3 / 24,595; IPC 0.767 / 0.615 (−20%)	Done
b5	4 cores × 4 workloads, fixed window 30M + 20M cycles	running

> **Finding:** the gem5 stdlib cache hierarchy (`PrivateL1SharedL2CacheHierarchy`) enables a StridePrefetcher at L1 and L2 by default. The baseline builds the caches manually without prefetchers so that miss rates reflect cache geometry only.

Workload generator "l2traffic" : one C++ program, key=value arguments, modes loop / stream / temporal / graph (plus stride, rand, chase and DSP-like kernels). Three mechanisms make results trustworthy: a **checksum** (host and gem5 must agree), a **region of interest** (`roi=1`: one warm-up pass, `m5_reset_stats`, measured passes, `m5_dump_stats`), and **"dump=N"**, which prints the exact address sequence used by the oracle.

3. Oracle v1 – Streaming Capacity Cliff

3.1 Why an oracle

- **Speed:** seconds per configuration instead of minutes on gem5.
- **Cross-checking:** two independent methods that agree validate each other; a misconfigured gem5 (prefetcher left on, wrong way count) would disagree.
- **Attributing causes:** the oracle models only data + LRU, so any gap with gem5 is caused by what it leaves out (code, stack, writebacks).

3.2 Design

```
address trace (l2traffic_native ... dump=N)  ->  L1D 64 sets x 8 ways, LRU
   -> L1 misses only ->  L2 1024 sets x W ways, LRU  ->  count (first pass excluded)
```

- Each set is an `OrderedDict`: hit → move to end; miss on a full set → evict the first entry (least recently used). This is true LRU, the same policy as gem5's `LRURP` (which evicts the block with the oldest last-touch tick).
- The first pass is excluded, matching `roi=1`, because cold misses would hide real differences between configurations.

3.3 Why streaming has a sharp cliff

A contiguous array of footprint F spreads evenly over the 1024 sets:

lines per set = F / 64 B / 1024 = F / 64 KiB

If lines per set ≤ W, everything fits and the miss rate after warm-up is 0%. If it exceeds W even by one, LRU in a cyclic sweep always evicts the line needed next, so every access misses (100%). All sets hold the same number of lines, so they fail together: the cliff is sharp.

3.4 Results

Oracle vs gem5 at stream 768 KiB: 16 ways 0% vs 0.02%; 8 ways 100% vs 100.00%; L2 accesses per pass 12,288 in both. The 8-way result at exactly 100% also confirms that gem5 runs without a prefetcher.

Sweep: 11 footprints × 5 way counts = 55 configurations, all following the rule miss = 0% if F ≤ 64 KiB × W, else 100%.

Run	Footprint	Ways	Point	Oracle	gem5	Diff
v_fp1024K_w16	1024K	16	exact fit	0.0%	1.13%	+1.1
v_fp1280K_w16	1280K	16	beyond	100.0%	100.00%	+0.0
v_fp512K_w8	512K	8	exact fit	0.0%	1.18%	+1.2
v_fp640K_w8	640K	8	beyond	100.0%	100.00%	+0.0
v_fp256K_w4	256K	4	exact fit	0.0%	1.29%	+1.3
v_fp384K_w4	384K	4	beyond	100.0%	100.00%	+0.0

Why ~1.2% at exact-fit points: in gem5 the L2 also holds code, stack and globals. Each foreign line in a full set creates W + 1 lines in a cycle, so all W data lines of that set miss. Hence miss ≈ k / 1024 for k polluted sets; 1.2% gives k ≈ 12 sets. W cancels out, which matches the nearly equal values at 16, 8 and 4 ways.

4. Oracle v2 – Stack Distance for All Workloads

4.1 Method

Because way shutdown keeps the number of sets fixed, one pass is enough. For each L2 access the oracle records the **stack distance**: the position of the line in its set's LRU list (0 = most recent). With W ways, the line is still present only if its distance is below W:

hit with W ways ⇔ distance < W miss(W) = accesses with distance ≥ W / all accesses

One histogram of distances therefore gives the miss rate for every W from 1 to 16. The result was verified to equal the direct LRU simulation (temporal512K: 9.38 / 27.22 / 63.66% at 16 / 8 / 4 ways from both methods). Run time for all workloads: about 3 seconds.

Workload	Arguments	Accesses / pass	Measured passes
loop768K	mode=loop footprint=768K	12,288	2
stream3x2M	mode=stream footprint=2M arrays=3	786,432	2
temporal512K	mode=temporal hot=512K cold=4M hotpct=90	73,728	4
graph2M	mode=graph footprint=2M	32,768	4

The trace length and warm-up match the gem5 runs exactly (one warm-up pass, `roi=1`), and the workloads use a fixed seed, so gem5 executes the same address sequence.

4.2 Results for the four sweep workloads and why

- **loop768K:** 768K / 64K = 12 lines per set → 0% down to 12 ways, 100% from 11 ways. Same mechanism as streaming.
- **stream3x2M:** 6 MiB of data, already larger than 16 ways → 100% at every way count. Way shutdown costs this workload nothing.
- **temporal512K:** a floor near 10% and a soft knee around 8 ways (explained in Section 5).
- **graph2M:** random pointer chasing over 2 MiB; the probability that the next vertex is still cached is roughly proportional to capacity, so the curve is close to linear, about +3.7 points per way removed.

5. Temporal Locality – Validation and Explanation

5.1 The workload

`mode=temporal hot=512K cold=4M hotpct=90`: each access picks a random line, 90% of the time from a 512 KiB hot region (8,192 lines) and 10% from a 4 MiB cold region (65,536 lines). Per L2 set this means **8 hot lines** and **64 cold lines**. A check on the dumped trace gave **90.0%** of accesses in the hot region, as designed.

5.2 gem5 validation

Ways	Oracle	gem5 miss	L2 accesses	DRAM reads	L2 MPKI	IPC
16	9.4%	9.36%	280,033	26,213	2.93	0.434
8	27.2%	27.22%	280,033	76,222	8.53	0.369 (−15%)
4	63.7%	63.63%	280,033	178,185	19.94	0.285 (−34%)

- Oracle and gem5 differ by less than 0.1 points. L2 accesses: 280,033 in gem5 vs 279,943 in the oracle (+90 from code/stack, 0.03%).
- DRAM reads equal L2 misses at every way count (e.g. 280,033 × 27.22% ≈ 76,225 vs 76,222): no prefetching and no extra traffic.
- All runs: checksum PASS, two stats blocks (ROI active), identical instruction count 8,937,201.
- The gap is much smaller than for streaming because temporal sets are never in an exactly-full state; a dozen foreign lines change only a handful of the ~280,000 accesses.

5.3 Why the curve looks like this

Floor ≈ 100 − hotpct. Cold accesses go to 64 lines per set with very rare reuse, so they almost always miss regardless of the number of ways. With hotpct = 90 the floor is about 10% (9.4% measured at 16 ways).

Knee ≈ hot / 64 KiB ways. With W well above 8, the 8 hot lines of each set stay resident with room for a few cold intruders. Near W = 8, each cold intruder pushes out a hot line. Below 8, the hot set no longer fits. Because accesses are random, eviction of hot lines is probabilistic, which makes the knee soft rather than a cliff.

5.4 Evidence: parameter variants (oracle)

If the explanation in 5.3 is right, changing the hot size should move the knee and changing hotpct should move the floor. Four variants were run on the oracle:

Variant	Hot lines/set	16w	12w	8w	6w	4w	Observation
hot 256K	4	9.3%	9.8%	10.7%	13.2%	27.6%	knee moves to ~4 ways
hot 512K	8	9.4%	11.5%	27.2%	44.3%	63.7%	knee ~8 ways
hot 768K	12	13.3%	26.8%	49.6%	62.5%	75.8%	knee moves to ~12 ways
hotpct 80	8	19.9%	26.9%	44.4%	57.3%	71.9%	floor rises to ~20%
hotpct 95	8	4.8%	5.2%	15.7%	36.3%	58.9%	floor drops to ~5%

Both predictions hold: the knee follows hot / 64 KiB and the floor follows 100 − hotpct.

5.5 Side finding: writebacks from initialization

Temporal is read-only, yet gem5 reported L2 writebacks in the ROI: 2,311 at 16 ways and 16 at 8 and 4 ways. These are dirty lines left over from **initializing** the 4.5 MiB buffer. With 8 or 4 ways they are flushed during the warm-up pass; with 16 ways many survive into the measured region. Miss rates are unaffected, but DRAM write counts at 16 ways are inflated (about 9% of DRAM traffic). **Lesson:** one warm-up pass is not enough to clean a large L2 when measuring write traffic; `warm=2` should be used for write-heavy workloads such as stream triad.

6. Implications for Way Shutdown (P1)

Workload type	Minimum ways to keep	Cost of going below
Streaming / loop (cyclic, footprint F)	[F / 64 KiB] + small margin for code/stack	jump to 100% miss (stream 768K: IPC −20% at 8 ways)
Streaming larger than L2	none	no extra cost for this workload; effect on other cores still to be measured
Temporal (hot H, hotpct p)	≈ H / 64 KiB + a few ways for cold intruders	rises quickly past the knee (temporal512K: +2.1 points 16→12 ways, +15.7 points 12→8)
Random graph (footprint G)	no knee; trade-off is linear	≈ +3.7 points per way removed for graph2M

7. Evidence Labels and Limitations

Label	Examples in this report
Measured (gem5)	b4 results; 6 streaming cliff points; temporal512K 9.36 / 27.22 / 63.63%; IPC and DRAM counts
Computed (oracle / formula)	55-configuration streaming table; curves for all workloads; temporal variants; 64 KiB × W rule
Assumed	L2 capacity 1 MiB; L1/L2/NoC latencies; L1 MSHR = 1
Out of scope	MSHR sweep

- The oracle has no code, stack or TLB (source of the ~1.2% gap at the streaming cliff).
- The oracle is single-threaded and does not model writebacks or latency, so it gives miss rates only, not AMAT.
- Ideal LRU only; the mentor's sweep also covers Random, FIFO, TreePLRU and NMRU, which the oracle cannot predict yet.
- IPC values depend on assumed latencies and are used only to show trends.

8. Next Steps (prepared)

1. Run gem5 for loop768K and graph2M at 16/8/4 ways (commands and compare_sd.py ready). Expected: loop 0 / 100 / 100%, graph 39.6 / 69.3 / 85.4%.
2. Run gem5 for stream3x2M with warm=2 to check DRAM write traffic.
3. Build a characterization table per workload (footprint, read/write ratio, stride, reuse distance, MLP) to justify the four workloads.
4. Extend the oracle to other replacement policies (TreePLRU, Random) and study multi-core interference.
5. Confirm latencies, L1 MSHR and L2 capacity with Tiến Anh.